components are suppressed by the factor (here) $\max(1 - 6f^2/f_c^2, 0)$. The second example is a sharpening filter where high frequencies are enhanced. Code for producing this image looks something like this:

rlft3_sharpen.h

```
Int i, j, nx=256, ny=256;                    Image is 256 × 256.
MatDoub data(nx,ny);
VecDoub speq(2*nx);
Doub fac;
...                                          Here we would fill data with the image.
rlft3(data,speq,1);                          Forward transform.
for (i=0;i<nx/2;i++) for (j=0;j<ny/2;j++) {      Loop over all frequencies ex-
        fac = 1.+3.*sqrt(SQR(i*2./nx)+SQR(j*2./ny));     cept Nyquist.
        Cmplx(data[i])[j] *= fac;
        if (i>0) Cmplx(data[nx-i])[j] *= fac;        Negative (wraparound) fre-
}                                                        quencies.
for (j=0;j<ny/2;j++) {                       Loop over frequencies where i is Nyquist.
    fac = 1.+3.*sqrt(1.+SQR(j*2./ny));
    Cmplx(data[nx/2])[j] *= fac;
}
for (i=0;i<nx/2;i++) {                       Loop over frequencies where j is Nyquist.
    fac = 1.+3.*sqrt(SQR(i*2./nx)+1.);
    Cmplx(speq)[i] *= fac;
    if (i>0) Cmplx(speq)[nx-i] *= fac;       Wraparound.
}
Cmplx(speq)[nx/2] *= (1.+3.*sqrt(2.));       Both i and j are Nyquist.
rlft3(data,speq,-1);                         Reverse transform.
```

The third example is a derivative filter, where a Fourier component at frequency (f_x, f_y) is multiplied by $2\pi i(f_x^2 + f_y^2)^{1/2}$, and the resulting intensities are then linearly mapped into an appropriate range.

To extend `rlft3` to four dimensions, you simply add an additional (outer) nested `for` loop in `i0`, analogous to the present `i1`. (Modifying the routine to do an *arbitrary* number of dimensions, as in `fourn`, is a good programming exercise for the reader.)

CITED REFERENCES AND FURTHER READING:

Brigham, E.O. 1974, *The Fast Fourier Transform* (Englewood Cliffs, NJ: Prentice-Hall).

Swartztrauber, P. N. 1986, "Symmetric FFTs," *Mathematics of Computation*, vol. 47, pp. 323–346.

Hutchinson, J. 2001, in *IEEE Professional Communication Society Newsletter*, vol. 45, no. 3. See also `http://www.lenna.org`.[1]

12.7 External Storage or Memory-Local FFTs

Sometime in your life, you might have to compute the Fourier transform of a *really large* data set, larger than the size of your computer's physical memory. In such a case, the data will be stored on some external medium, such as magnetic or optical disk. Needed is an algorithm that makes some manageable number of sequential passes through the external data, processing it on the fly and outputting intermediate results to other external media, which can be read on subsequent passes.

In fact, an algorithm of just this description was developed by Singleton [1] very soon after the discovery of the FFT. The algorithm requires four sequential storage devices, each

capable of holding half of the input data. The first half of the input data is initially on one device, the second half on another.

Singleton's algorithm is based on the observation that it is possible to bit reverse 2^M values by the following sequence of operations: On the first pass, values are read alternately from the two input devices, and written to a single output device (until it holds half the data), and then to the other output device. On the second pass, the output devices become input devices, and vice versa. Now, we copy *two* values from the first device, then *two* values from the second, writing them (as before) first to fill one output device, then to fill a second. Subsequent passes read 4, 8, etc., input values at a time. After completion of pass $M - 1$, the data are in bit-reverse order.

Singleton's next observation is that it is possible to alternate the passes of essentially this bit-reversal technique with passes that implement one stage of the Danielson-Lanczos combination formula (12.2.3). The scheme, roughly, is this: One starts as before with half the input data on one device and half on another. In the first pass, one complex value is read from each input device. Two combinations are formed, and one is written to each of two output devices. After this "computing" pass, the devices are rewound, and a "permutation" pass is performed, where groups of values are read from the first input device and alternately written to the first and second output devices; when the first input device is exhausted, the second is similarly processed. This sequence of computing and permutation passes is repeated $M-K-1$ times, where 2^K is the size of internal buffer available to the program. The second phase of the computation consists of a final K computation passes. What distinguishes the second phase from the first is that, now, the permutations are local enough to do in place during the computation. There are thus no separate permutation passes in the second phase. In all, there are $2M - K - 2$ passes through the data.

An implementation of Singleton's algorithm, `fourfs`, based on reference [1], is given in a Webnote [2].

For one-dimensional data, Singleton's algorithm produces output in exactly the same order as a standard FFT (e.g., `four1`). For multidimensional data, the output is in *transpose* order rather than in the conventional C++ array order output by `fourn`. That is, in scanning through the data, it is the leftmost array index that cycles most quickly, then the second leftmost, and so on. This peculiarity, which is intrinsic to the method, is generally only a minor inconvenience. For convolutions, one simply computes the component-by-component product of two transforms in their nonstandard arrangement, and then does an inverse transform on the result. Note that, if the lengths of the different dimensions are not all the same, then you must reverse the order of the values in `nn[0..ndim-1]` (thus giving the dimensions of the transpose-order output array) before performing the inverse transform. Note also that, just like `fourn`, performing a transform and then an inverse results in multiplying the original data by the product of the lengths of all dimensions.

We leave it as an exercise for the reader to figure out how to reorder `fourfs`'s output into normal order, taking additional passes through the externally stored data. We doubt that such reordering is ever really needed.

You will likely want to modify `fourfs` to fit your particular application. For example, as written, `KBF` $\equiv 2^K$ plays the dual role of being the size of the internal buffers, and the record size of the unformatted reads and writes. The latter role limits its size to that allowed by your machine's I/O facility. It is a simple matter to perform multiple reads for a much larger `KBF`, thus reducing the number of passes by a few.

Another modification of `fourfs` would be for the case where your virtual memory machine has sufficient address space, but not sufficient physical memory, to do an efficient FFT by the conventional algorithm (whose memory references are extremely nonlocal). In that case, you will need to replace the reads, writes, and rewinds by mappings of the arrays `afa`, `afb`, and `afc` into your address space. In other words, these arrays are replaced by references to a single data array, with offsets that get modified wherever `fourfs` performs an I/O operation. The resulting algorithm will have its memory references local within blocks of size `KBF`. Execution speed is thereby sometimes increased enormously, albeit at the cost of requiring twice as much virtual memory as an in-place FFT.

CITED REFERENCES AND FURTHER READING:

Singleton, R.C. 1967, "A Method for Computing the Fast Fourier Transform with Auxiliary Memory and Limited High-speed Storage," *IEEE Transactions on Audio and Electroacoustics*, vol. AU-15, pp. 91–97.[1]

Numerical Recipes Software 2007, "Code for External or Memory-Local Fourier Transform," *Numerical Recipes Webnote No. 18*, at `http://www.nr.com/webnotes?18` [2]

Oppenheim, A.V., Schafer, R.W., and Buck, J.R. 1999, *Discrete-Time Signal Processing*, 2nd ed. (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.